Nama : Nadya Aurelly Putri
NIM : 255150700111036
Kelas : TI – D
Asisten Praktikum : 1. Daffa Ramadhan Jauhari
2. Alya Hamidah
Modul : 6

1. Soal 1

Jalankan code program diatas dan benahi jika menemukan kesalahan!

A. Kode

```
import java.util.*;
public class Employee {
    private String name;
    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int
month, int day){
        this.name = name;
        this.salary = salary;
        GregorianCalendar calendar = new
GregorianCalendar(year,month-1,day);
        this.hireday = calendar.getTime();
    }
    public Date getHireDay(){
        return hireday;
    }
    public String getName(){
        return name;
    }
    public double getSalary(){
        return salary;
    }
    public void raiseSalary(double byPercent){
        double raise = salary * byPercent/100;
        salary+=raise;
    }
```

```
}
public class Manager extends Employee {
    private double bonus;
    public Manager(String name, double salary, int year, int
month, int day){
        super(name, salary, year, month, day);
        bonus = 0;
    }
    public void setBonus(double bonus){
        this.bonus = bonus;
    }
    public double getSalary(){
        double baseSalary = super.getSalary();
        return baseSalary+bonus;
    }

}
public class MainEmployee {
    public static void main(String[] args) {
        Manager boss = new Manager("Steven", 80000, 1987, 12, 15);
        boss.setBonus(5000);

        Employee staff = new Employee("Donni", 50000, 1989, 10, 1);

        System.out.println("nama boss : " + boss.getName() + ", salary
= " + boss.getSalary());
        System.out.println("nama staff : " + staff.getName() + ",
salary = " + staff.getSalary());
    }
}
```

B. Penjelasan

Setelah program dijalankan, tidak ditemukan error atau kesalahan, sehingga program dapat berjalan dengan baik.

C. Hasil Foto

```
nama boss : Steven, salary = 85000.0
nama staff : Donni, salary = 50000.0
PS D:\Pemrograman Lanjut>
```

1. Soal 2

Bagaimana cara konstruktor pada subclass memanggil konstruktor di superclass nya? Apakah hal itu perlu dilakukan? Sertakan alasan anda !

A. Kode

B. Penjelasan

```
Konstruktor pada subclass memanggil konstruktor di superclass
menggunakan keyword super(), kebanyakan ditulis di baris
pertama dalam konstruktor subclass.

Ini perlu dilakukan karena superclass memiliki atribut atau
inisialisasi yang harus dijalankan terlebih dahulu sebelum
subclass digunakan. Dengan super(), data dari superclass
dapat terisi dengan benar sehingga objek subclass dapat
bekerja secara lengkap dan tidak menimbulkan error.
```

C. Hasil Foto

-

1. Soal 3

Tambahkan constructor pada class Employee dengan parameter String name! amati perubahan apa yang terjadi, jelaskan jawaban anda!

A. Kode

```
import java.util.*;
public class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }
```

B. Penjelasan

Setelah ditambahkan constructor dengan parameter String name pada class Employee, tidak terjadi error dan program tetap dapat dijalankan seperti sebelumnya. Hal ini karena constructor yang baru hanya menambah variasi (overloading) dan tidak menghapus constructor yang sudah ada.

C. Hasil Foto

```
nama boss : Steven, salary = 85000.0
nama staff : Donni, salary = 50000.0
```

1. Soal 4

Pada Class Manager baris ke 5, setelah variable day tambahkan variable bonus! Amati apa yang terjadi dan mengapa demikian?

A. Kode

```
import java.util.*;
public class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }
```

B. Penjelasan

Setelah ditambahkan variable bonus, program menjadi error, karena variabel tersebut belum diinisialisasi dan tidak termasuk parameter pada constructor superclass. Selain itu, constructor pada superclass tidak memiliki parameter bonus, sehingga menyebabkan ketidaksesuaian argumen.

C. Hasil Foto

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Cannot refer to an instance field bonus while explicitly invoking a constructor

        at Manager.<init>(Manager.java:4)
        at MainEmployee.main(MainEmployee.java:3)
```

1. Soal 5

Untuk apa digunakan keyword this pada class manager dan employee?

Hapus keyword this dan amati apa yang terjadi?

A. Kode

```
public class Manager extends Employee {
    private double bonus;
    public Manager(String name, double salary, int year, int month,
int day){
        super(name, salary, year, month, day);
        bonus = 0;
    }
    public void setBonus(double bonus){
        bonus = bonus;
import java.util.*;
public class Employee {
    private String name;

    public Employee(String name){
        name = name;
    }
    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int
```

```
month, int day){
        name = name;
        salary = salary;
        GregorianCalendar calendar = new
GregorianCalendar(year,month-1,day);
        hireday = calendar.getTime();
    }
```

B. Penjelasan

Keyword this digunakan untuk mengakses dan membedakan atribut milik objek dengan parameter atau variabel lokal, sehingga tidak terjadi kebingungan dalam penulisan kode. Setelah keyword this pada class employee dan manager dihapus, tidak terjadi error, namun nama boss dan staff menjadi null dan nilai salary menjadi 0.0, karena tanpa penggunaan this, nilai parameter tidak disimpan ke dalam atribut (instance variable) milik objek. Yang terjadi adalah parameter hanya mengisi variabel lokal saja, sehingga atribut name dan salary pada objek tetap memiliki nilai default, yaitu null untuk String dan 0.0 untuk double

C. Hasil Foto

```
nama boss : null, salary = 0.0
nama staff : null, salary = 0.0
```

1. Soal 6

Tambahkan constructor pada class Employee dengan parameter Bertipe data string bernama name yang nantinya bila constructor ini akan dipanggil akan menginisialisasi variable name! Amati perubahannya pada class anak dan jelaskan! Benahi bila terjadi kesalahan!

A. Kode

```
import java.util.*;
public class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }
    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int month,
int day){
        this.name = name;
        this.salary = salary;
        GregorianCalendar calendar = new
GregorianCalendar(year,month-1,day);
        this.hireday = calendar.getTime();
    }
    public Date getHireDay(){
        return hireday;
    }
    // constructor baru
    public Employee(String name) {
        this.name = name;
// setelah dibenahi (pada class manager)
public class Manager extends Employee {
    private double bonus;
    public Manager(String name, double salary, int year, int month,
int day){
        super(name);
        bonus = 0;
    }
```

B. Penjelasan

Setelah ditambahkan constructor Employee(String name) pada class Employee dan constructor lama dihapus, program menjadi error. Hal ini terjadi karena pada class anak (Manager) masih memanggil constructor lama menggunakan super(name, salary, year, month, day), sedangkan constructor tersebut sudah tidak tersedia di superclass.

Untuk memperbaiki kesalahan tersebut, pemanggilan constructor pada subclass harus disesuaikan dengan constructor yang ada di superclass, yaitu:

```
 public Manager(String name) {

super(name);

}
```

C. Hasil Foto

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        Duplicate method Employee(String) in type Employee
        Duplicate method Employee(String) in type Employee
        Syntax error, insert "}" to complete MethodBody

        at Employee.<init>(Employee.java:5)
        at Manager.<init>(Manager.java:4)
        at MainEmployee.main(MainEmployee.java:3)
```

Setelah dibenahi:

```
nama boss : Steven, salary = 5000.0
nama staff : Donni, salary = 50000.0
PS D:\Pemrograman Lanjut>
```

1. Soal 4

Pada Class Manager baris ke 5, setelah variable day tambahkan variable bonus! Amati apa yang terjadi dan mengapa demikian?

A. Kode

```
import java.util.*;
public class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }
```

B. Penjelasan

```
Setelah ditambahkan variable bonus, program menjadi error,
karena variabel tersebut belum diinisialisasi dan tidak
termasuk parameter pada constructor superclass. Selain itu,
constructor pada superclass tidak memiliki parameter bonus,
sehingga menyebabkan ketidaksesuaian argumen.
```

C. Hasil Foto

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Cannot refer to an instance field bonus while explicitly invoking a constructor

        at Manager.<init>(Manager.java:4)
        at MainEmployee.main(MainEmployee.java:3)
```

1. Soal 7

Pada bab sebelumnya anda telah belajar mengenai konsep encapsulation, jelaskan mengapa pada super class menggunakan modifier protected? Apa yang terjadi jika modifier anda ubah menjadi private atau public? Jelaskan !

A. Kode

```
-
```

B. Penjelasan

```
Pada superclass digunakan modifier protected supaya atribut
atau method masih bisa diakses oleh class turunan (subclass),
tapi tetap tidak bebas diakses dari luar class seperti
public. Jadi, protected ini semacam "jalan tengah" antara
keamanan dan fleksibilitas.

Kalau diubah jadi private, maka atribut atau method tersebut
tidak bisa diakses langsung oleh subclass, sehingga class
turunan harus memakai getter/setter. Ini lebih aman
```

(encapsulation lebih kuat), tapi jadi kurang fleksibel saat pewarisan.

Kalau diubah jadi public, maka atribut bisa diakses dari mana saja, termasuk dari luar class. Ini memang lebih mudah, tapi mengurangi keamanan data, karena siapa pun bisa mengubah nilainya tanpa kontrol.

C. Hasil Foto

-

1. Soal 8

Ubahlah acces modifier method pada kelas employee menjadi :

a. Private

b. Protected

Amati perubahan apa yang terjadi? Jelaskan jawaban anda dengan detail!

A. Kode

Sebelum diubah:

```
import java.util.*;
public class Employee {
    private String name;

    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int month,
int day){
        this.name = name;
        this.salary = salary;
        GregorianCalendar calendar = new
```

```
GregorianCalendar(year,month-1,day);
        this.hireday = calendar.getTime();
    }
    public Date getHireDay(){
        return hireday;
    }
    public Employee(String name) {
        this.name = name;
    }

    public String getName(){
        return name;
    }
    public double getSalary(){
        return salary;
    }
    public void raiseSalary(double byPercent){
        double raise = salary * byPercent/100;
        salary+=raise;
    }
}
```

Setelah diubah jadi private:

```
import java.util.*;
public class Employee {
    private String name;

    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int month,
int day){
        this.name = name;
        this.salary = salary;
        GregorianCalendar calendar = new
GregorianCalendar(year,month-1,day);
        this.hireday = calendar.getTime();
    }
    private Date getHireDay(){
        return hireday;
    }
```

```
    public Employee(String name) {
        this.name = name;
    }

    private String getName(){
        return name;
    }
    private double getSalary(){
        return salary;
    }
    public void raiseSalary(double byPercent){
        double raise = salary * byPercent/100;
        salary+=raise;
    }
}
```

Setelah diubah jadi protected:

```
import java.util.*;
public class Employee {
    private String name;

    private double salary;
    private Date hireday;
    public Employee(String name, double salary, int year,int month,
int day){
        this.name = name;
        this.salary = salary;
        GregorianCalendar calendar = new
GregorianCalendar(year,month-1,day);
        this.hireday = calendar.getTime();
    }
    protected Date getHireDay(){
        return hireday;
    }
    public Employee(String name) {
        this.name = name;
    }

    protected String getName(){
        return name;
```

```
    }
    protected double getSalary(){
        return salary;
    }
    public void raiseSalary(double byPercent){
        double raise = salary * byPercent/100;
        salary+=raise;
    }
}
```

B. Penjelasan

Pada superclass digunakan modifier protected supaya atribut atau method masih bisa diakses oleh class turunan (subclass), tapi tetap tidak bebas diakses dari luar class seperti public. Jadi, protected ini semacam "jalan tengah" antara keamanan dan fleksibilitas.

Kalau diubah jadi private, maka atribut atau method tersebut tidak bisa diakses langsung oleh subclass, sehingga class turunan harus memakai getter/setter. Ini lebih aman (encapsulation lebih kuat), tapi jadi kurang fleksibel saat pewarisan.

Kalau diubah jadi public, maka atribut bisa diakses dari mana saja, termasuk dari luar class. Ini memang lebih mudah, tapi mengurangi keamanan data, karena siapa pun bisa mengubah nilainya tanpa kontrol.

C. Hasil Foto

Setelah diubah jadi private:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        The method getName() from the type Employee is not visible
        The method getName() from the type Employee is not visible
        The method getSalary() from the type Employee is not visible

        at MainEmployee.main(MainEmployee.java:8)
```

Setelah diubah jadi protected:

```
nama boss : Steven, salary = 5000.0
nama staff : Donni, salary = 50000.0
PS D:\Pemrograman Lanjut>
```

TUGAS PRAKTIKUM

Soal:

Susunlah program sesuai studi kasus di bawah ini:

1. Manusia.java

   Kelas manusia merupakan kelas induk dengan definisi sebagai berikut:

   - nama : String
   - jenisKelamin : boolean (true : laki-laki, false : perempuan)
   - nik : String
   - menikah : boolean
   + setter, getter
   + getTunjangan() : double
   + getPendapatan() : double
   + toString() : String

   (Keterangan)

   - Tunjangan untuk yang telah menikah adalah apabila laki-laki akan mendapat $25 sedangkan perempuan mendapat $20.
   - Tunjangan untuk yang belum menikah adalah $15 .
   - toString() menampilkan nama, nik, jenis kelamin, dan jumlah pendapatan.

2. MahasiswaFILKOM.java

   Kelas mahasiswa merupakan kelas turunan dari Manusia dengan definisi sebagai berikut:

   - nim : String
   - ipk : double
   + setter, getter
   + getStatus() : String
   + getBeasiswa() : double
   + toString() : String

   (Keterangan)

   - Beasiswa untuk ipk 3.0 – 3.5 mendapat $50 dan untuk 3.5 – 4 mendapat $75
   - Status untuk mendapatkan angkatan dan prodi (menurut kaidah FILKOM UB)

     1651506XXXXXXX

     - Digit ke 1-2 adalah angkatan
     - Digit ke 7 adalah prodi
       - 2 Teknik Informatika
       - 3 Teknik Komputer
       - 4 Sistem Informasi
       - 6 Pendidikan Teknologi Informasi
       - 7 Teknologi Informasi

     Dengan pengembalian dengan format : prodi angkatan, contoh : Sistem Informasi, 2017
   - toString() menampilkan atribut induk + nim, ipk, dan status.

3. Pekerja.java
   Kelas Pekerja merupakan kelas turunan dari Manusia dengan definisi sebagai berikut:
   - gaji : double
   - tahunMasuk : LocalDate
   - jumlahAnak : int
   + setter, getter
   + getBonus() : double
   + getGaji() : double
   + toString() : String

   (Keterangan)
   - Bonus didapatkan oleh pegawai sesuai lama bekerja :
     - Jika lama bekerja 0 – 5 tahun, maka bonus sebesar 5% dari gaji
     - Jika lama bekerja 5 – 10 tahun, maka bonus sebesar 10% dari gaji
     - Jika lebih dari 10 tahun, maka bonus sebesar 15% dari gaji
   - Tunjangan ditambah apabila memiliki anak, yaitu $20 per anak.
- toString() menampilkan atribut induk + tahun masuk, jumlah anak, dan gaji.

4. Manager.java
   Kelas Manager merupakan kelas turunan dari Pekerja dengan definisi sebagai berikut:
   - departemen : String
   + setter, getter

   (Keterangan)
   - Tunjangan ditambah sebesar 10% dari gaji.
   - toString() menampilkan atribut induk + departemen.

Dari semua kelas yang telah dibuat, buatlah testcase (mencetak objek / toString()) untuk kasus berikut :

1. Manusia
   a. Laki-laki telah menikah.
   b. Perempuan telah menikah.
   c. Belum menikah.
2. MahasiswaFilkom (sesuai status Anda)
   a. Ipk < 3
   b. Ipk 3 – 3.5
   c. Ipk 3.5 – 4
3. Pekerja
   a. Lama bekerja 2 tahun, anak 2
   b. Lama bekerja 9 tahun
   c. Lama bekerja 20 tahun, anak 10
4. Manager, lama bekerja 15 tahun dengan gaji $7500

Contoh Input testcase – output
*(*Tidak perlu dimasukkan laporan*)*

`Manusia a = new Manusia("A", "111", true, true);` `System.out.println(a);`	nama : A nik : 111 jenisKelamin : Laki-laki pendapatan : 25.0
`MahasiswaFILKOM b = new MahasiswaFILKOM("165150300111100", 4.0, "B", "111", false, false);` `System.out.println(b);`	nama : B nik : 111 jenisKelamin : Perempuan pendapatan : 90.0 nim : 165150300111100 ipk : 4.0 status : Teknik Komputer, 2016
`Pekerja c = new Pekerja(1000, 2016, 3, 2, 4, "C", "111", true, true);` `System.out.println(c);`	nama : C nik : 111 jenisKelamin : Laki-laki pendapatan : 1155.0 tahun masuk : 2 3 2016 jumlah anak : 4 gaji : 1000.0
`Manager d = new Manager("HRD", 1000, 2017, 1, 2, 3, "D", "111", true, true);` `System.out.println(d);`	nama : D nik : 111 jenisKelamin : Laki-laki pendapatan : 1235.0 tahun masuk : 2 1 2017 jumlah anak : 3 gaji : 1000.0 departemen : HRD

A.Kode

```
public class Manusia {
    private String nama;
    boolean jenisKelamin;
    private String nik;
    private boolean menikah;

    public Manusia(){

    }

     public Manusia(String nama, boolean jenisKelamin, String nik,
boolean menikah){
        this.nama = nama;
```

```
        this.jenisKelamin = jenisKelamin;
        this.nik = nik;
        this.menikah = menikah;
    }

    public double getTunjangan(){
        if (menikah){
            if(jenisKelamin){
                return 25;//laki
            } else {
                return 20;//perempuan
            }
        }
        return 15;//belum menikah
    }

    public double getPendapataN(){
        return getTunjangan();
    }
    public double getPendapatan(){
        return getTunjangan();
    }

   @Override
    public String toString(){
        return "Nama           : " + nama +
           "\nNIK            : " + nik +
           "\nJenis Kelamin   : " + (jenisKelamin ? "Laki-laki" :
"Perempuan") +
           "\nPendapatan      : " + getPendapatan();


    }
 }

public class mahasiswaFILKOM extends Manusia {
    private String nim;
    private double ipk;

    public mahasiswaFILKOM(String nama, boolean jenisKelamin, String
```

```
nik, boolean menikah, String nim, double ipk){
        super(nama, jenisKelamin, nik, menikah);
        this.nim = nim;
        this.ipk = ipk;
    }

    public double getBeasiswa(){
        if(ipk >= 3.0 && ipk <= 3.5){
            return 50;
        } else if(ipk > 3.5 && ipk <= 4.0){
            return 75;
        }
        return 0;
    }

    public String getStatus(){
        String angkatan = "20" + nim.substring(0, 2);
        char kode = nim.charAt(6);

        String prodi = "";
        switch(kode){
            case '2': prodi = "Teknik Informatika"; break;
            case '3': prodi = "Teknik Komputer"; break;
            case '4': prodi = "Sistem Informasi"; break;
            case '6': prodi = "Pendidikan Teknologi Informasi"; break;
            case '7': prodi = "Teknologi Informasi"; break;
        }

        return prodi + ", " + angkatan;
    }

    @Override
    public double getPendapatan(){
        return super.getPendapatan() + getBeasiswa();
    }

    @Override
    public String toString(){
        return super.toString() + "\n" +
               "NIM             : " + nim + "\n" +
```

```
                "IPK            : " + ipk + "\n" +
                "Status         : " + getStatus() + "\n" +
                "Beasiswa       : " + getBeasiswa();
    }
}

import java.time.LocalDate;
import java.time.Period;

public class Pekerja extends Manusia {
    private double gaji;
    private LocalDate tahunMasuk;
    private int jumlahAnak;

     public Pekerja(double gaji, int tahun, int bulan, int hari, int
jumlahAnak,
                   String nama, String nik, boolean jenisKelamin,
boolean menikah){

        super(nama, jenisKelamin, nik, menikah);
        this.gaji = gaji;
        this.tahunMasuk = LocalDate.of(tahun, bulan, hari);
        this.jumlahAnak = jumlahAnak;
    }
    public double getBonus(){
        int lamaKerja = Period.between(tahunMasuk,
LocalDate.now()).getYears();

        if(lamaKerja >= 0 && lamaKerja <= 5){
            return 0.05 * gaji;
        } else if(lamaKerja >=6 && lamaKerja <= 10){
            return 0.10 * gaji;
        } else {
            return 0.15 * gaji;
        }
    }

    public double getGaji(){
        return gaji + getBonus() + (jumlahAnak * 20);
    }
```

```
    @Override
    public double getPendapatan(){
        return super.getPendapatan() + getGaji();
    }

   @Override
public String toString(){
    return super.toString() + "\n" +
          "tahun masuk     : " +
          tahunMasuk.getDayOfMonth() + " " +
          tahunMasuk.getMonthValue() + " " +
          tahunMasuk.getYear() + "\n" +
           "jumlah anak     : " + jumlahAnak + "\n" +
           "gaji            : " + gaji;
   }
}

public class Manager extends Pekerja {
    private String departemen;

    public Manager(String departemen, double gaji, int tahun, int
bulan, int hari, int jumlahAnak,
                   String nama, String nik, boolean jenisKelamin,
boolean menikah){

        super(gaji, tahun, bulan, hari, jumlahAnak, nama, nik,
jenisKelamin, menikah);
        this.departemen = departemen;
    }

    @Override
    public double getTunjangan(){
        return super.getTunjangan() + (0.10 * super.getGaji());
    }

     @Override
    public String toString(){
        return super.toString() + "\n" +
               "departemen      : " + departemen;
```

```
    }
}
public class MainFilkom {
    public static void main(String[] args) {
        Manusia a = new Manusia("A", true, "111", true);
        System.out.println(a);

        mahasiswaFILKOM b = new mahasiswaFILKOM("B", false, "111",
false, "165150300111100", 4.0);
        System.out.println("\n" + b);

        Pekerja c = new Pekerja(1000, 2016, 3, 2, 4, "C", "111", true,
true);
        System.out.println("\n" + c);

        Manager d = new Manager("HRD", 1000, 2017, 2, 3, 3, "D",
"111", true, true);
        System.out.println("\n" + d);
    }
}
```

B.Penjelasan

Pada program ini, terdapat beberapa class yang saling berhubungan menggunakan konsepinheritance. Class yang paling dasar adalah Class Manusia. Class ini digunakan untuk menyimpan data umum seperti nama, jenis kelamin, NIK, dan status menikah. Selain itu, di dalam class ini juga terdapat method untuk menghitung tunjangan. Besarnya tunjangan ditentukan berdasarkan status menikah dan jenis kelamin. Jika sudah menikah, maka laki-laki mendapat tunjangan lebih besar dibanding perempuan, sedangkan jika belum menikah, tunjangannya lebih kecil. Nilai tunjangan ini nantinya juga digunakan sebagai dasar perhitungan pendapatan.

Selanjutnya ada Class mahasiswaFILKOM yang merupakan turunan dari class Manusia. Class ini menambahkan atribut baru yaitu NIM dan IPK. Di sini, mahasiswa bisa mendapatkan beasiswa tergantung dari IPK yang dimiliki. Semakin tinggi IPK, maka beasiswa yang didapat juga lebih besar. Selain itu, terdapat method untuk menentukan status mahasiswa, yang diambil dari NIM, seperti angkatan dan program studi. Pendapatan pada

mahasiswa dihitung dari tunjangan yang diwarisi dari class Manusia ditambah dengan beasiswa yang didapatkan.

Kemudian ada Class Pekerja, yang juga merupakan turunan dari class Manusia. Class ini digunakan untuk merepresentasikan seorang pegawai. Atribut yang ditambahkan antara lain gaji, tanggal mulai bekerja, dan jumlah anak. Dalam class ini terdapat perhitungan bonus yang didasarkan pada lama bekerja. Semakin lama bekerja, maka bonus yang diperoleh semakin besar. Selain itu, jumlah anak juga mempengaruhi total pendapatan karena ada tambahan tunjangan per anak. Jadi, pendapatan pekerja merupakan gabungan dari tunjangan dasar, gaji, bonus, dan tunjangan anak.

Berikutnya adalah Class Manager, yang merupakan turunan dari class Pekerja. Class ini menambahkan atribut departemen. Seorang manager memiliki keuntungan tambahan berupa tunjangan khusus sebesar persentase tertentu dari gaji. Dengan demikian, pendapatan manager menjadi lebih besar karena mencakup tunjangan dasar, gaji, bonus, tunjangan anak, serta tambahan tunjangan sebagai manager.

Terakhir, pada Class MainFilkom, program dijalankan dengan membuat beberapa objek dari masing-masing class, yaitu Manusia, MahasiswaFILKOM, Pekerja, dan Manager. Setiap objek diisi dengan data yang berbeda, kemudian ditampilkan menggunakan method toString().

C. Hasil Foto

```
Nama           : A
NIK            : 111
Jenis Kelamin  : Laki-laki
Pendapatan     : 25.0

Nama           : B
NIK            : 111
Jenis Kelamin  : Perempuan
Pendapatan     : 90.0
NIM            : 165150300111100
IPK            : 4.0
Status         : Teknik Komputer, 2016
Beasiswa       : 75.0

Nama           : C
NIK            : 111
Jenis Kelamin  : Laki-laki
Pendapatan     : 1205.0
tahun masuk    : 2 3 2016
jumlah anak    : 4
gaji           : 1000.0

Nama           : D
NIK            : 111
Jenis Kelamin  : Laki-laki
Pendapatan     : 1301.0
tahun masuk    : 3 2 2017
jumlah anak    : 3
gaji           : 1000.0
departemen     : HRD
```